

AI Lab assignment 3

Tejas Patil

U24AI061

1. Implement a Simple Reflex Agent for a vacuum cleaner that can navigate a three-room environment (A, B, and C) and maintain cleanliness. Identify the “rationality” of its actions. How will you define the performance cost? Define the complete Rule Table and store it either in a dictionary/vector/matrix based on the language you implement the vacuum cleaner. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

TYPE I

```
import random

# Environment
rooms = {'A': 'Dirt', 'B': 'Dirt', 'C': 'Dirt'}
location = 'A'

# Rule Table
rule_table = {
    ('A', 'Dirt'): ['Remove'],
    ('A', 'No Dirt'): ['Move Right'],
    ('B', 'Dirt'): ['Remove'],
    ('B', 'No Dirt'): ['Move Left', 'Move Right'],
    ('C', 'Dirt'): ['Remove'],
    ('C', 'No Dirt'): ['Move Left']
}

def move(loc, action):
    if loc == 'A' and action == 'Move Right': return 'B'
    if loc == 'B' and action == 'Move Left': return 'A'
    if loc == 'B' and action == 'Move Right': return 'C'
    if loc == 'C' and action == 'Move Left': return 'B'
```

```

        return loc

print("Percept\t\tAction\t\tLocation")
print("-----")

for _ in range(7):
    status = rooms[location]
    actions = rule_table[(location, status)]
    action = random.choice(actions)

    print(f"({location},{status})\t{action}\t\t{location}")

    if action == 'Remove':
        rooms[location] = 'No Dirt'
    else:
        location = move(location, action)

```

```

C:\Users\tejas\OneDrive\Desktop\AI Lab\Assignment 3>python -u "c:\Users\tejas\OneDrive\Desktop\AI L
ab\Assignment 3\Q1.py"
Percept      Action      Location
-----
(A,Dirt)      Remove      A
(A,No Dirt)   Move Right   A
(B,Dirt)      Remove      B
(B,No Dirt)   Move Right   B
(C,Dirt)      Remove      C
(C,No Dirt)   Move Left    C
(B,No Dirt)   Move Right   B

```

TYPE II

```

import random

def vacuum_agent(location, status):
    if status == "Dirt":
        return "Remove"
    if location == "A":
        return "Move Right"
    if location == "C":
        return "Move Left"
    if location == "B":
        return random.choice(["Move Left", "Move Right"])

```

```

steps = int(input("Enter number of percepts: "))

print("\nPercept\t\tAction")
print("-----")

for _ in range(steps):
    location = input("Enter Location (A/B/C): ")
    status = input("Enter Status (Dirt/No Dirt): ")

    action = vacuum_agent(location, status)
    print(f"({location}, {status})\t{action}")

```

```

Percept      Action
-----
Enter Location (A/B/C): A
Enter Status (Dirt/No Dirt): Dirt
(A, Dirt)      Remove
Enter Location (A/B/C): B
Enter Status (Dirt/No Dirt): No Dirt
(B, No Dirt)   Move Left

```

2. Design a Simple Reflex Agent for an Indian Railways Level Crossing. The agent must ensure maximum safety for road traffic while minimizing delays for the train, using multi-source sensory input without historical tracking. The environment consists of a double-track railway line intersecting a busy state highway. The percepts include the following:

TYPE I:

```

def railway_agent(percept):
    emergency, train_in, train_out, obstacle = percept

    # Priority 1: Emergency
    if emergency == "Active":
        return ("Lower", "On", "Red")

    # Priority 2: Obstacle
    if obstacle == "Blocked":

```

```

        return ("Lower", "On", "Red")

    # Priority 3: Train detected
    if train_in == "Detected" or train_out == "Detected":
        return ("Lower", "On", "Green")

    # Priority 4: Normal
    return ("Raise", "Off", "Green")

test_cases = [
    ("Neutral", "Detected", "NotDetected", "Clear"),
    ("Neutral", "NotDetected", "NotDetected", "Blocked"),
    ("Active", "NotDetected", "NotDetected", "Clear"),
    ("Neutral", "NotDetected", "NotDetected", "Clear")
]

print("Percepts\t\t\t\t\tAction (Gate, Hooter, Signal)")
print("-----")

for p in test_cases:
    action = railway_agent(p)
    print(f"{p}\t\t\t\t\t{action}")

```

```

Percepts                                Action (Gate, Hooter, Signal)
-----
('Neutral', 'Detected', 'NotDetected', 'Clear') ('Lower', 'On', 'Green')
('Neutral', 'NotDetected', 'NotDetected', 'Blocked') ('Lower', 'On', 'Red')
('Active', 'NotDetected', 'NotDetected', 'Clear') ('Lower', 'On', 'Red')
('Neutral', 'NotDetected', 'NotDetected', 'Clear') ('Raise', 'Off', 'Green')

```

TYPE II:

```

def railway_agent(emergency, train_in, train_out, obstacle):
    # Priority 1: Emergency
    if emergency == "Active":
        return ("Lower", "On", "Red")

    # Priority 2: Obstacle
    if obstacle == "Blocked":
        return ("Lower", "On", "Red")

```

```

# Priority 3: Train detected
if train_in == "Detected" or train_out == "Detected":
    return ("Lower", "On", "Green")

# Priority 4: Normal
return ("Raise", "Off", "Green")

steps = int(input("Enter number of percepts: "))

print("\nPercepts\t\t\t\t\tAction (Gate, Hooter, Signal)")
print("-----")
)

for _ in range(steps):
    emergency = input("Manual Emergency (Active/Neutral): ")
    train_in = input("Train Inbound (Detected/NotDetected): ")
    train_out = input("Train Outbound (Detected/NotDetected): ")
    obstacle = input("Obstacle Sensor (Blocked/Clear): ")

    action = railway_agent(emergency, train_in, train_out, obstacle)
    print(f"({emergency}, {train_in}, {train_out},
{obstacle})\t{action}\n")

```

Percepts	Action (Gate, Hooter, Signal)

Manual Emergency (Active/Neutral): Active	
Train Inbound (Detected/NotDetected): Detected	
Train Outbound (Detected/NotDetected): Detected	
Obstacle Sensor (Blocked/Clear): Clear	
(Active, Detected, Detected, Clear)	('Lower', 'On', 'Red')